

Module 3

Advanced Queries, Joins & Windows

Turn related rows into accurate reports while preserving the meaning of every result.

QUERY

04

related
tables

One row grain
at a time

By the end, you can build explainable multi-table reports

1 COMBINE

Use INNER and LEFT joins to connect customers, orders, products and order lines without accidental row loss.

2 SUMMARIZE

Use GROUP BY, aggregate functions, HAVING and FILTER at a clearly stated business grain.

3 ANALYZE

Use subqueries, CTEs and window functions for rankings, comparisons and running totals.

Success is visible: every report has a stated grain, validated row count and defensible total.

The lesson moves from row meaning to analytical output

01 Frame

Define output grain and SQL processing order

15 min

02 Join

Combine tables and control unmatched rows

20 min

03 Analyze

Aggregate, filter, rank and compare

25 min

04 Hands-on

Build a four-query reporting pack

30 min

Checkpoint

Explain why each output row exists and how every total was calculated.

State the result grain before writing the SELECT list

ONE ROW PER PRODUCT

Suitable for a product catalogue or current-stock report. Product is the grain.

ONE ROW PER ORDER

Suitable for an order total. Line items must be grouped back to the order grain.

ONE ROW PER CUSTOMER

Suitable for lifetime value. Orders and items are summarized to the customer grain.

Most wrong totals begin with an unstated grain or an unnoticed one-to-many join.

Add a few completed orders before running the reports

```
INSERT INTO app.customers(email, full_name) VALUES  
('ben@example.com','Ben Lee'),  
('chen@example.com','Chen Wei')  
ON CONFLICT (email) DO NOTHING;  
  
INSERT INTO app.products(sku,name,price,stock_qty) VALUES  
('HD-300','USB-C Hub',129.00,18),  
('ST-400','Laptop Stand',159.00,12)  
ON CONFLICT (sku) DO NOTHING;
```

MINIMUM DATA

Use at least three customers, four products, four orders and six order lines.

INCLUDE A GAP

Keep one customer without an order and one product never sold. LEFT JOIN will reveal both.

Run `SELECT count(*)` against each table and record the starting counts.

SQL is written in one order but evaluated in another

1 FROM + JOIN

Build the working row set

2 WHERE

Remove individual rows

3 GROUP BY

Form groups at the chosen grain

4 HAVING

Remove entire groups

5 SELECT

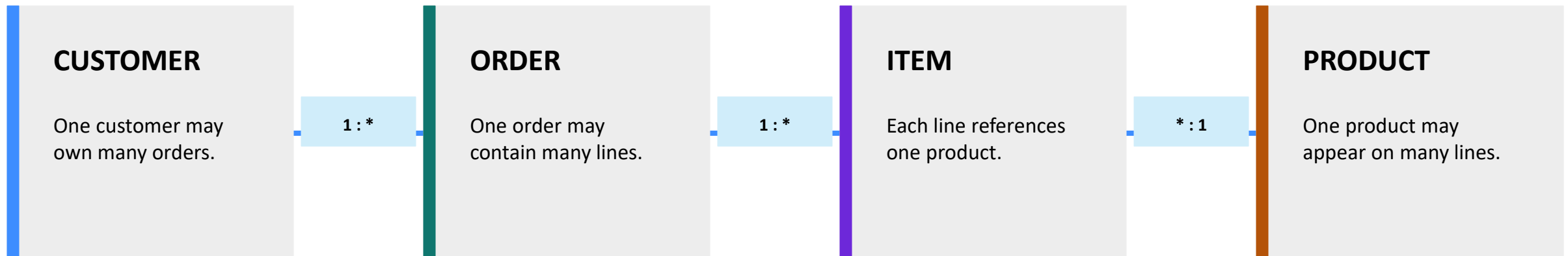
Calculate and name output columns

6 ORDER BY

Present the final rows

This explains why an aggregate alias is usually unavailable in WHERE.

A join expands rows according to matching relationships



Joining order to item changes the grain from one row per order to one row per order line.

Aggregate only after confirming this row multiplication is intentional.

INNER JOIN returns only rows with a valid match

```
SELECT o.order_id,  
       c.full_name,  
       o.status,  
       o.created_at  
FROM app.orders AS o  
JOIN app.customers AS c  
  ON c.customer_id = o.customer_id  
ORDER BY o.order_id;
```

RESULT GRAIN

One row per order because every order matches exactly one customer.

USE ALIASES

Short aliases keep qualification readable and prevent ambiguous-column errors.

Use an explicit ON condition so the relationship is visible during review.

LEFT JOIN preserves the rows that have no match

```
SELECT c.customer_id,  
       c.full_name,  
       count(o.order_id) AS order_count  
FROM app.customers AS c  
LEFT JOIN app.orders AS o  
  ON o.customer_id = c.customer_id  
GROUP BY c.customer_id, c.full_name  
ORDER BY order_count DESC;
```

WHY COUNT THE KEY?

count(o.order_id) returns 0 for an unmatched customer; count(*) would count the preserved row.

NULL MEANS NO MATCH

Columns from the right-hand table are NULL when no order exists.

LEFT JOIN answers “include everyone, even when activity is absent.”

A four-table join reveals each purchased product line

```
SELECT o.order_id,  
       c.full_name,  
       p.sku,  
       i.quantity,  
       i.unit_price,  
       i.quantity * i.unit_price AS line_total  
FROM app.orders AS o  
JOIN app.customers AS c USING (customer_id)  
JOIN app.order_items AS i USING (order_id)  
JOIN app.products AS p USING (product_id)  
ORDER BY o.order_id, p.sku;
```

RESULT GRAIN

One row per order item—not one row per order or customer.

CALCULATED VALUE

line_total is computed from the historical unit price stored on the item.

Aggregate functions summarize rows at a chosen grain

`count(*)`

How many input rows?

`count(column)`

How many non-NULL values?

`sum(value)`

What is the total?

`avg(value)`

What is the arithmetic mean?

`min / max`

What are the boundaries?

Every selected non-aggregate column must identify the group or be functionally dependent on it.

WHERE filters rows; HAVING filters completed groups

```
SELECT o.customer_id,
       count(*) AS paid_orders,
       sum(i.quantity * i.unit_price) AS paid_value
FROM app.orders AS o
JOIN app.order_items AS i USING (order_id)
WHERE o.status = 'PAID'
GROUP BY o.customer_id
HAVING sum(i.quantity * i.unit_price) >= 300;
```

WHERE

Removes unpaid order rows before grouping and calculation.

HAVING

Keeps only customer groups whose calculated value reaches 300.

Filter as early as meaning allows; group only the rows the report needs.

Group order lines back to one row per order

```
SELECT o.order_id,  
       c.full_name,  
       o.status,  
       sum(i.quantity * i.unit_price) AS order_total  
FROM app.orders AS o  
JOIN app.customers AS c USING (customer_id)  
JOIN app.order_items AS i USING (order_id)  
GROUP BY o.order_id, c.full_name, o.status  
ORDER BY order_total DESC;
```

BEFORE GROUPING

The working set contains one row per order line.

AFTER GROUPING

The output contains one row per order with a summed total.

Validate one order manually by summing its displayed line totals.

FILTER produces several targeted metrics in one grouping

```
SELECT c.customer_id,  
       c.full_name,  
       count(o.order_id) AS all_orders,  
       count(o.order_id) FILTER  
         (WHERE o.status = 'PAID') AS paid_orders,  
       count(o.order_id) FILTER  
         (WHERE o.status = 'CANCELLED') AS cancelled_orders  
FROM app.customers AS c  
LEFT JOIN app.orders AS o USING (customer_id)  
GROUP BY c.customer_id, c.full_name;
```

ONE GROUPING PASS

Each aggregate sees only the rows allowed by its own FILTER condition.

KEEP ZEROES

LEFT JOIN preserves customers with no orders so their metrics remain visible.

EXISTS answers a relationship question without multiplying rows

```
SELECT p.product_id, p.sku, p.name
FROM app.products AS p
WHERE EXISTS (
  SELECT 1
  FROM app.order_items AS i
  JOIN app.orders AS o USING (order_id)
  WHERE i.product_id = p.product_id
  AND o.status = 'PAID'
)
ORDER BY p.sku;
```

QUESTION

Has this product appeared in at least one paid order?

BEHAVIOR

PostgreSQL can stop searching after the first qualifying match.

Use **NOT EXISTS** for “products never ordered” without NULL pitfalls.

A CTE gives a complex intermediate result a clear name

```
WITH order_totals AS (  
  SELECT o.order_id,  
         o.customer_id,  
         sum(i.quantity * i.unit_price) AS total  
  FROM app.orders AS o  
  JOIN app.order_items AS i USING (order_id)  
  GROUP BY o.order_id, o.customer_id  
)  
SELECT c.full_name,  
       count(*) AS order_count,  
       sum(t.total) AS lifetime_value  
FROM order_totals AS t  
JOIN app.customers AS c USING (customer_id)  
GROUP BY c.customer_id, c.full_name;
```

STEP 1

order_totals establishes one row per order.

STEP 2

The outer query summarizes those orders to one row per customer.

Window functions calculate across rows without collapsing them

GROUP BY

Many input rows become fewer output rows.

Use when the report needs one row per group.

WINDOW

Every input row remains visible while a related calculation is added.

Use for ranks and comparisons.

OVER (...)

PARTITION BY creates independent groups; ORDER BY establishes sequence.

Ask first: should detail rows disappear? If no, consider a window function.

Rank products by units sold

```
WITH product_sales AS (  
  SELECT p.product_id, p.sku,  
         sum(i.quantity) AS units_sold  
  FROM app.products AS p  
  JOIN app.order_items AS i USING (product_id)  
  GROUP BY p.product_id, p.sku  
)  
SELECT sku,  
       units_sold,  
       dense_rank() OVER (  
         ORDER BY units_sold DESC  
       ) AS sales_rank  
FROM product_sales  
ORDER BY sales_rank, sku;
```

DENSE_RANK

Ties share a rank and the following rank has no gap.

ALTERNATIVES

row_number gives unique sequence;
rank leaves gaps after ties.

A running total adds sequence-aware context to each order

```
WITH order_totals AS (  
  SELECT o.order_id, o.customer_id, o.created_at,  
         sum(i.quantity * i.unit_price) AS total  
  FROM app.orders AS o  
  JOIN app.order_items AS i USING (order_id)  
  GROUP BY o.order_id, o.customer_id, o.created_at  
)  
SELECT order_id, customer_id, created_at, total,  
       sum(total) OVER (  
         PARTITION BY customer_id  
         ORDER BY created_at, order_id  
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
       ) AS running_value  
FROM order_totals;
```

PARTITION

Restart the calculation for each customer.

FRAME

Include all earlier rows through the current row in deterministic order.

Hands-on 3: build a management reporting pack

A	Order totals	One row per order with customer, status and calculated total.
---	---------------------	---

B	Customer activity	Include every customer, even when order_count is zero.
---	--------------------------	--

C	Product ranking	Rank products by units sold; show ties consistently.
---	------------------------	--

D	Running value	Show each customer order with cumulative spending.
---	----------------------	--

For every query, write its grain and expected row count before execution.

Validate meaning—not only successful execution

1 Row count

Does the output contain the expected number of business entities?

2 Manual total

Does one order total equal the sum of its displayed line totals?

3 Unmatched row

Does the customer without orders appear with count zero?

4 Window behavior

Do ranks and running totals preserve every detail row?

Common warning signs: duplicate totals, missing zero-activity rows, ambiguous columns and unstable ranking order.

Knowledge check: explain what each clause protects

1

Why can a correct join still produce a wrong total?

2

When should LEFT JOIN replace INNER JOIN?

3

What is the difference between HAVING and WHERE?

4

Why does a window function preserve more detail than GROUP BY?

Pair-share: support every answer with one query from the hands-on reporting pack.

Your reports now preserve grain, matches and sequence

COMBINE

Joins connect related business rows.

SUMMARIZE

Aggregates return the intended report grain.

ANALYZE

Windows add rank and context without losing detail.

Next: Module 4 — Indexes, transactions and concurrency